RAG-Powered Pandas Chatbot

with Hybrid Q&A Retrieval

Retrieval-Augmented Generation · AGAI-03 Project · Agentic AI Bootcamp

Field	Detail
Course / Batch	Agentic AI Bootcamp – AGAI-03
Project	Website-Specific RAG Chatbot using Scraping + Hybrid Retrieval
Target Website	pandas.pydata.org/docs/user_guide
LLM (Q&A gen & chatbot)	Ollama · llama3.1:8b (local, offline)
Embeddings	multi-qa-MiniLM-L6-cos-v1 (sentence-transformers)
Framework	LangChain · ChromaDB · Streamlit
Date	29 May 2026

Table of Contents

1. Introduction & Website Choice

1.1 Project Overview

This project implements a Retrieval-Augmented Generation (RAG) chatbot that answers questions about the pandas Python library by retrieving relevant documentation passages and generating accurate, grounded responses via a locally running large language model. The system combines two retrieval strategies, direct Q&A matching and semantic document search, with a conversational LLM to deliver precise answers with source citations. The entire pipeline runs offline using Ollama; no cloud API keys are required.

1.2 Chosen Website

Target: https://pandas.pydata.org/docs/user_guide/ · Pages scraped: 23 pages across the full User Guide · Content type: Official Python library documentation (HTML, Sphinx-based).

The following User Guide sections were scraped:

#	Section	Slug / Filename
1	10 Minutes to pandas	10min.html
2	Intro to Data Structures	dsintro.html
3	Essential Basic Functionality	basics.html
4	IO Tools (read_csv, to_excel …)	io.html
5	Indexing & Selecting Data	indexing.html
6	MultiIndex / Advanced Indexing	advanced.html
7	Merge, Join, Concatenate	merging.html
8	Reshaping & Pivot Tables	reshaping.html
9	Working with Text Data	text.html
10	Working with Missing Data	missing_data.html
11	Duplicate Labels	duplicates.html
12	Categorical Data	categorical.html
13	Chart Visualisation	visualization.html
14	Group By: Split-Apply-Combine	groupby.html
15	Windowing Operations	window.html
16	Time Series / Date Functionality	timeseries.html
17	Time Deltas	timedeltas.html
18	Options & Settings	options.html
19	Enhancing Performance	enhancingperf.html
20	Scaling to Large Datasets	scale.html

21	Sparse Data Structures	sparse.html
22	Frequently Asked Questions (Gotchas)	gotchas.html
23	Cookbook	cookbook.html

1.3 Reasons for Choosing Pandas Documentation

- **High query frequency:** pandas is among the most-used Python packages; developers and data scientists constantly look up API usage, making a domain-specific Q&A chatbot immediately useful.
- **Structured content:** Sphinx-generated HTML has a consistent layout (headings, code blocks, parameter tables), which simplifies extraction of clean, coherent text chunks.
- **Rich knowledge density:** Each page contains numerous functions, parameters, and code examples – ideal for generating diverse, specific Q&A pairs.
- **Publicly accessible documentation:** The pandas documentation is publicly available and well suited for educational experiments and RAG development projects.
- **Factual grounding requirement:** Library documentation demands precise, factual answers (parameter names, return types), making it a good application of RAG, where retrieved documentation can help reduce hallucinations and improve answer accuracy.

2. Scraping Methodology & Challenges

2.1 Tools & Libraries

The scraper (src/scraper.py) uses requests for HTTP fetching and BeautifulSoup 4 with the lxml parser for HTML parsing. The pipeline is:

– Iterate over a predefined list of 23 target URLs from config.py

– Fetch each page with a polite User-Agent header identifying the bot.

– Parse the page and remove non-content elements (navigation, footer, sidebar, search box).

– Extract the main article content using PyData Sphinx Theme selectors with a fallback chain.

– Unwrap inline formatting, convert code blocks to readable lines, and render tables as pipe-separated text.

– Clean and normalise whitespace, then save as a plain .txt file with a metadata header.

– Wait SCRAPE_DELAY = 2 seconds between requests to respect the server.

2.2 HTML Cleaning Strategy

The pandas documentation uses the PyData Sphinx Theme. Content is located via a cascading selector chain: div.bd-content → article.bd-article → div#main-content → div.body → main. Before text extraction the scraper decomposes these non-content elements:

- nav, footer, script, style – structural chrome
- .headerlink – permalink anchors (¶ symbols)
- .toctree-wrapper, #searchbox, .sphinxsidebar – navigation panels
- .bd-sidebar, .bd-toc, .prev-next-area – theme UI elements

Inline tags (em, strong, a, span, inline code) are unwrapped so text flows naturally; <pre> code blocks are converted to single 'Code: …' lines; headings h2–h6 are prefixed with '## ' so the Q&A generator can later split on section boundaries. Excessive blank lines are collapsed to at most one. Each saved file begins with a metadata header:

```
URL: https://pandas.pydata.org/docs/user_guide/groupby.html
TITLE: Group by: split-apply-combine
SCRAPED_AT: 2026-05-21T10:34:12
------------------------------------------------------------
(content follows)
```

2.3 Challenges Faced

Challenge	Description	Resolution
PyData Sphinx Theme selectors	The modern PyData theme nests content differently from classic Sphinx; a single CSS selector did not cover all pages.	Implemented a cascading selector chain (bd-content → bd-article → main → body) with fallback.
Navigation / sidebar noise	Without removing sidebar elements, extracted text contained irrelevant menu items, increasing Q&A noise.	Explicit tag.decompose() calls for all non-content elements before text extraction.
Header-link artefacts	Sphinx inserts ¶ permalink symbols after every heading via .headerlink tags.	Removed via CSS selector before text extraction.
Rate limiting / politeness	Scraping 23 pages too quickly risks IP blocks and loads the server.	Added a 2-second delay between requests and a descriptive User-Agent string.

3. Q&A Generation

3.1 Overview

The Q&A generator (src/qa_generator.py) converts raw scraped text into structured question-answer pairs using a locally running LLM. It operates in a fully offline, rate-limit-free manner using Ollama with the llama3.1:8b model. The final dataset contains 1347 Q&A pairs saved to data/processed/qa_dataset_ollama.csv.

3.2 Model Choice

Model: llama3.1:8b via Ollama (local inference). The model runs entirely offline – no API keys, no rate limits, no cost per token. llama3.1:8b was chosen because it produces coherent JSON-structured Q&A pairs with sufficient instruction-following for structured output.

3.3 Prompt Design

The prompt instructs the model to act as a practical dataset generator for data-science practitioners, to write general and standalone questions (not tied to example variables), and to return strict JSON. The exact prompt used in code is:

```
You are building a Q&A dataset for a RAG chatbot that helps coders, data scientists,
data analysts, and data engineers use the pandas library effectively. Your goal is to
generate question-answer pairs that teach users HOW to use pandas in real data
workflows — not just what a function is called. Given the pandas documentation below,
generate exactly {n} question-answer pairs.

RULES:

- Questions must be general and standalone — a user must be able to ask them WITHOUT
having read the docs

- NEVER reference "the example", "the code", or example variable names (df, df2, s, ...)

- Questions must be practical: purpose, how to use it, key parameters, return value, or
common use cases

- Answers must be explanatory and include a short code example when relevant

Return ONLY a valid JSON object with a "pairs" key: {"pairs":
[{"question": "...", "answer": "..."}, ...]}

TEXT:
{chunk}
```

3.4 Chunking Strategy

Before passing content to the LLM, each page is split into chunks of max 1,500 characters. The algorithm:

- Splits first on section headings (the '## ' markers added by the scraper), keeping related content together.
- Sections that still exceed 1,500 chars are split further on paragraph boundaries, then by sentence if a single paragraph is still too long.
- Chunks shorter than 100 characters are filtered out (too short to generate meaningful pairs).

Each chunk requests a variable number of pairs – n = max(2, min(5, len(chunk) // 300)) – so longer chunks yield up to 5 pairs and short chunks yield at least 2. This balances diversity with speed on CPU inference.

3.5 JSON Robustness

- **Markdown fence stripping:** Removes ```json / ``` wrappers if the model adds them.
- **Retry logic (max 2 retries):** On a JSON parse failure the same chunk is retried up to 2 more times.
- **Graceful skip:** After 3 failed attempts the chunk is skipped with a log message.
- **Cleanup before parsing:** trailing commas removed, smart quotes normalized, unescaped newlines collapsed.
- **Field validation:** only pairs containing both a 'question' and an 'answer' key are kept.

3.6 Incremental & Resumable Writing

Generated pairs are flushed to the CSV every three pairs (streaming append), so most progress is preserved if generation is interrupted. On restart the generator reads the existing CSV and skips any source page that has already been processed (resume-by-source page), avoiding duplicate work across runs.

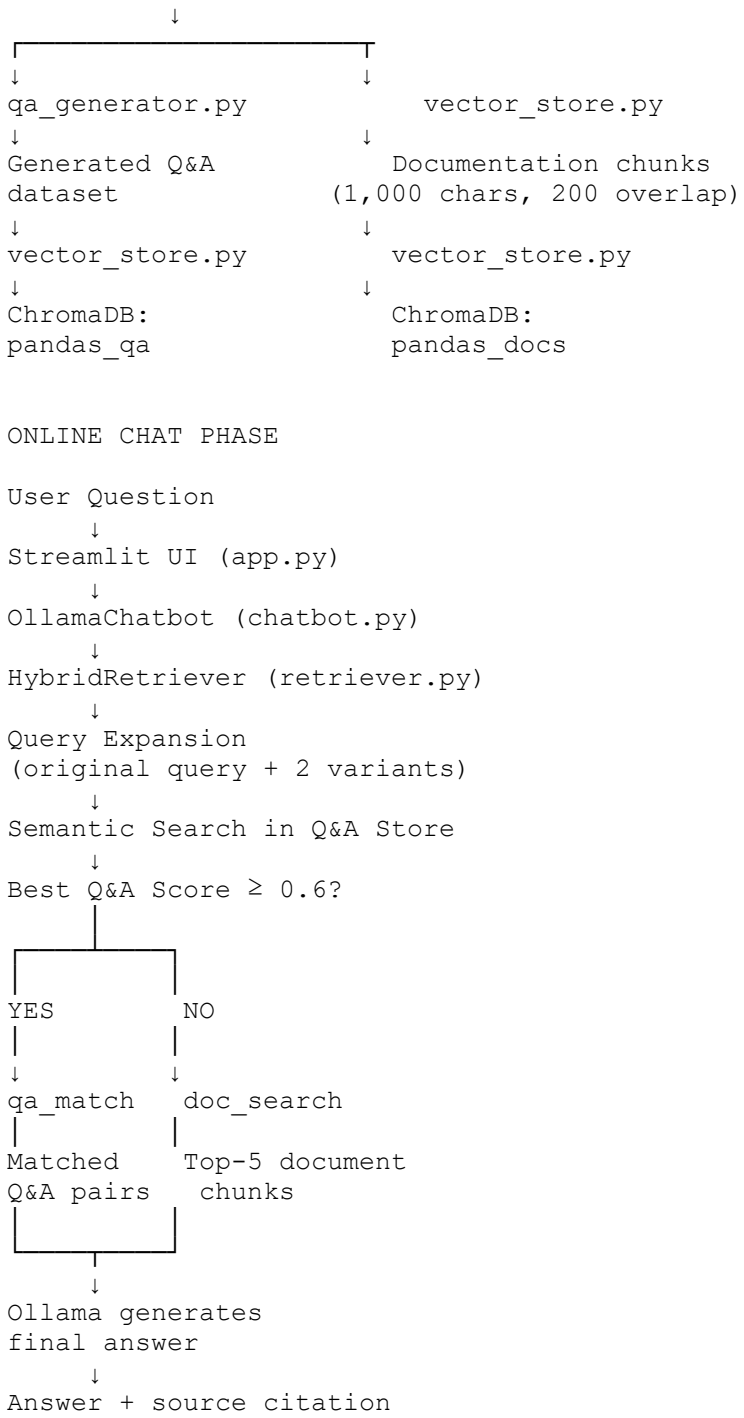
3.7 Pipeline Summary (Scraping+QA Generation)

Step	Action	Config
1	Load 23 .txt files from data/raw/	–
2	Split each page into chunks	max_chars = 1,500
3	POST each chunk to Ollama /api/chat	model = llama3.1:8b
4	Parse JSON response, validate fields	max_retries = 2
5	Collect valid pairs per chunk	n = max(2, min(5, len(chunk) // 300))
6	Append to CSV (resumable)	flush every three pairs
7	Output dataset	data/processed/qa_dataset_ollama.csv

4. System Architecture

4.1 Full Pipeline Diagram

```
Pandas User Guide URLs (23 pages)
    ↓
scraper.py
    ↓
Cleaned text files
data/raw/*.txt
```



4.2 Component Summary

Component	File	Technology	Role
Web Scraper	src/scraper.py	requests, BeautifulSoup4, lxml	Downloads & cleans pandas docs
Q&A Generator	src/qa_generator.py	Ollama REST API, llama3.1:8b	Creates structured Q&A dataset
Vector Store	src/vector_store.py	ChromaDB, sentence-transformers	Embeds & indexes docs and Q&A pairs
Hybrid Retriever	src/retriever.py	LangChain, ChromaDB, Ollama	Decides retrieval strategy per query

Chatbot	src/chatbot.py	Ollama, llama3.1:8b	Generates answers with context & memory
UI	app.py	Streamlit	Web interface for user interaction
Config	config.py	python-dotenv	Central configuration

5. Hybrid Retrieval Explanation

5.1 Motivation

A RAG system must balance precision (returning the exact right answer) and coverage (handling questions not in the Q&A dataset). Pure Q&A matching is fast and precise but fails on novel phrasing. Pure document search is flexible but forces the LLM to synthesise answers from long contexts, risking hallucination. The hybrid approach uses a threshold-based decision gate: if the user's question closely matches known Q&A pairs, those pairs are used as high-confidence context; otherwise the system falls back to semantic document retrieval.

5.2 Two-Store Architecture

Store	Collection	Content	Embedding
Q&A Store	pandas_qa	Questions embedded; answer + source URL stored as metadata	multi-qa-MiniLM-L6-cos-v1
Doc Store	pandas_docs	Raw documentation chunks (1,000 characters, 200 overlap)	multi-qa-MiniLM-L6-cos-v1

Document chunks are embedded using multi-qa-MiniLM-L6-cos-v1 and stored in ChromaDB for semantic retrieval.

5.3 Retrieval Decision Logic

1. **User Query**
   The user submits a question through the Streamlit interface.
2. **Query Expansion**
   The original query is sent to Ollama, which generates two documentation-style reformulations. This helps improve retrieval recall when users phrase questions colloquially or differently from the wording used in the official documentation.
3. **Q&A Store Search**
   The original query and its reformulations are searched against the Q&A vector store. The system retrieves the top-3 matches and keeps the highest relevance score.
4. **Q&A Match Path**
   If the best relevance score is greater than or equal to **0.60**, the system enters **qa_match mode**. The retrieved Q&A pairs are provided as context to Ollama, which synthesises the final answer. If the language model is unavailable, the stored answer can be returned as a fallback.
5. **Document Search Path**
   If the best relevance score is below **0.60**, the system enters **doc_search mode**. The document vector store is searched, and the top-5 most relevant documentation chunks are retrieved and combined into a context window. Ollama then generates a grounded answer based on the retrieved documentation.

6. **Out-of-Scope Detection**
   If the best document relevance score is below **0.20** and the query is not identified as a follow-up question, the system returns a polite refusal message rather than generating a potentially hallucinated answer.

5.4 Threshold Choice (0.6)

The threshold of 0.6 was set empirically during testing and provided a good balance between retrieving relevant Q&A matches and avoiding incorrect matches. The threshold is configurable via HYBRID_THRESHOLD in config.py. Scores above the threshold are treated as Q&A matches, while lower scores trigger document-based retrieval.

5.5 Multi-Query Expansion

Before searching the Q&A store, the retriever asks Ollama to rephrase the user's question into two documentation-style variants (temperature 0, capped at 80 tokens). The original query and both variants are searched, and the best relevance score across all three is used for the threshold decision. This improves recall when a user's colloquial phrasing ("how do I read a csv") differs from the formally-worded stored question. If Ollama is unavailable the expansion is skipped silently and only the original query is used.

5.6 Embedding Model

- Optimised specifically for semantic similarity / question-answer retrieval tasks.
- Compact (~80 MB) – fast to load and run on CPU.
- 384-dimensional dense vectors – good quality for technical-domain search.
- Runs fully locally – no external API calls for embeddings.

5.7 Conversation Memory

The chatbot maintains a rolling window of the last 6 conversation turns (MAX_HISTORY_TURNS). Previous turns are included in the LLM message list to resolve follow-up references such as "it" or "that". Short follow-up queries without pandas keywords trigger a retrieval rewrite that prepends the previous user topic. Memory is held in-process and can be reset via the 'Clear Chat' button.

6. Chatbot Interface & Screenshots

6.1 Interface Overview

The chatbot is served as a Streamlit web app (app.py), accessible at http://localhost:8501 after running 'streamlit run app.py'. The layout consists of a sidebar with metadata and a main chat area. The app checks that Ollama is running before allowing any queries.

6.2 Sidebar Features

- **Ollama status:** live connection indicator showing the connected model (llama3.1:8b).
- **Statistics:** pages scraped, Q&A pairs, doc chunks, and current memory turns.
- **Sample questions:** one-click example queries to demonstrate chatbot capabilities.
- **Q&A preview:** expandable random sample of Q&A pairs from the dataset.
- **About panel:** hybrid retrieval strategy, model info, and vector-DB details.
- **Clear Chat:** resets conversation memory.

6.3 Chat Interface Features

- Standard chat-bubble layout (user on the right, assistant on the left).
- Retrieval-mode badge: 'Q&A Match (xx%)' in green or 'Doc Search (xx%)' in blue.

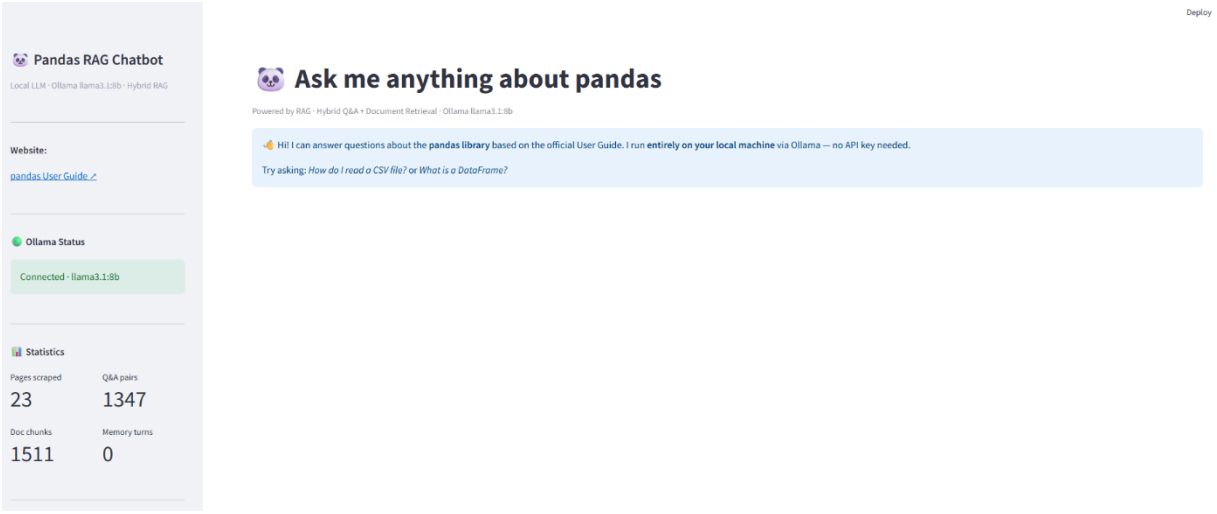
- Source URL citation at the bottom of each response.
- Out-of-scope handling: questions unrelated to pandas receive a polite refusal.
- Matched Q&A expander: shows the matched question from the dataset in qa_match mode.
- Streaming responses: the answer is rendered token-by-token as the LLM generates it.

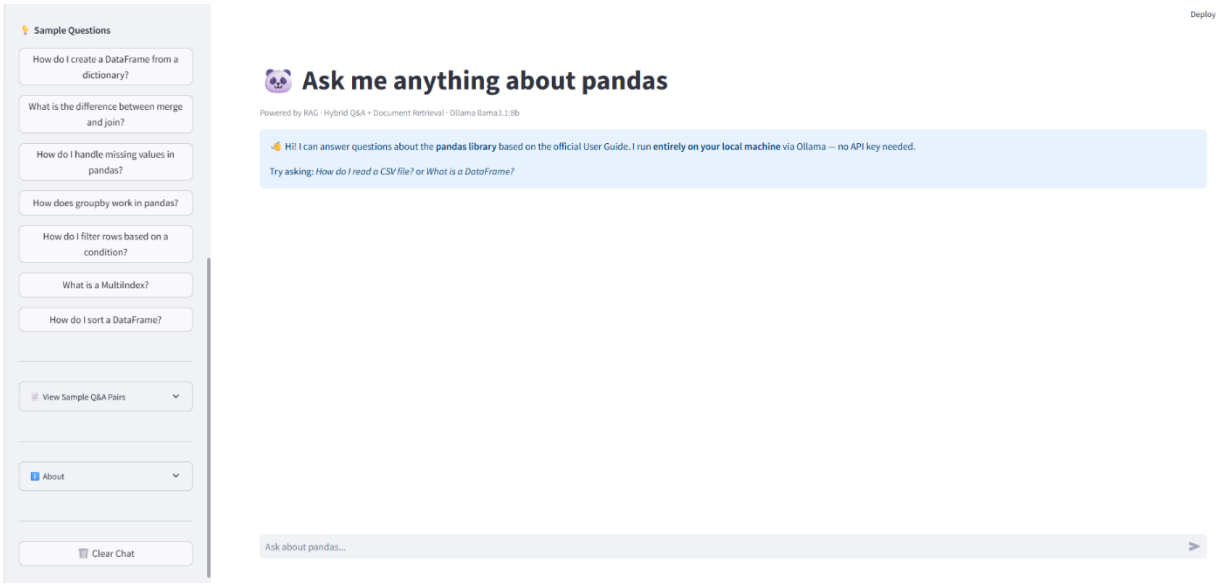
6.4 Screenshots

The following screenshots demonstrate the chatbot in operation. All were captured from the running Streamlit application (app.py) connected to Ollama llama3.1:8b.

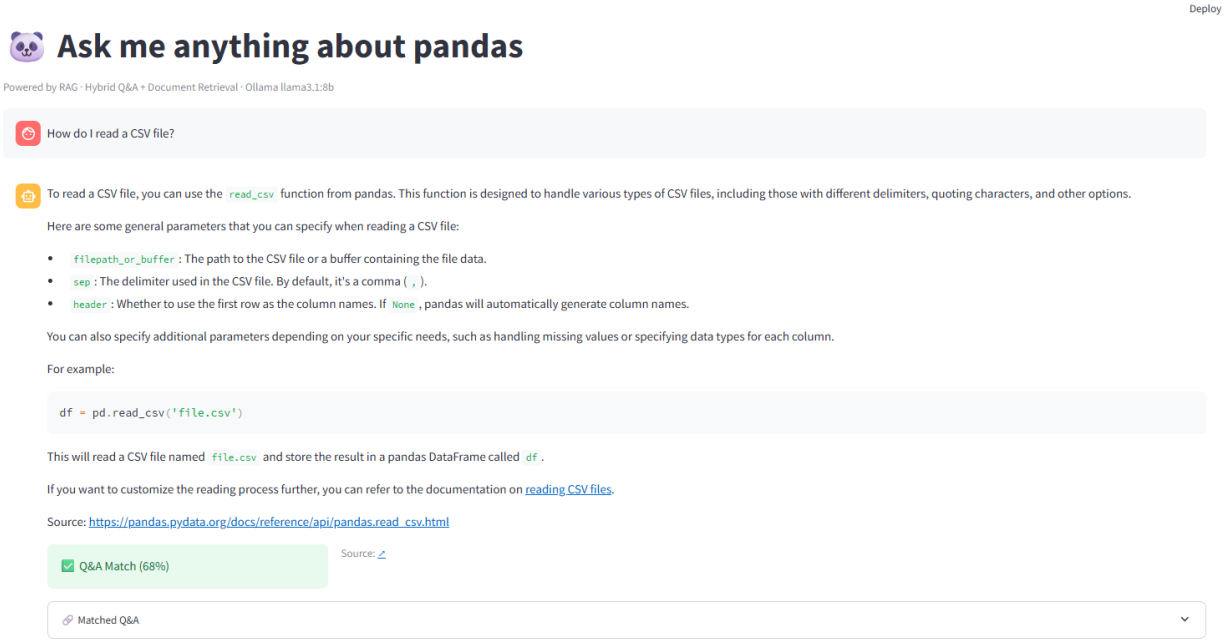
Screenshot 1 and 2 – Full chatbot UI

The main interface showing the sidebar with Ollama connected status (llama3.1:8b), statistics, sample-question buttons, and the welcome message in the main chat area.

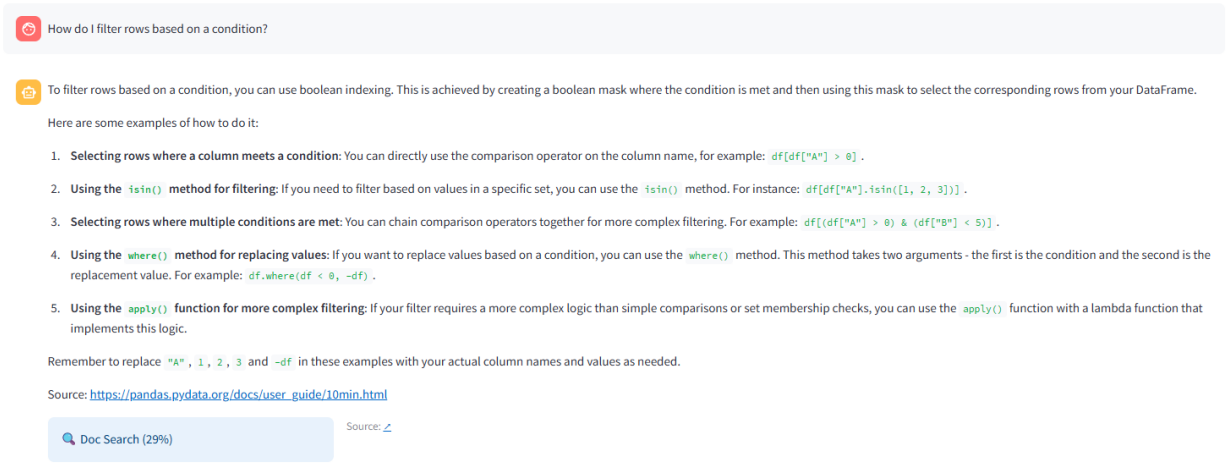




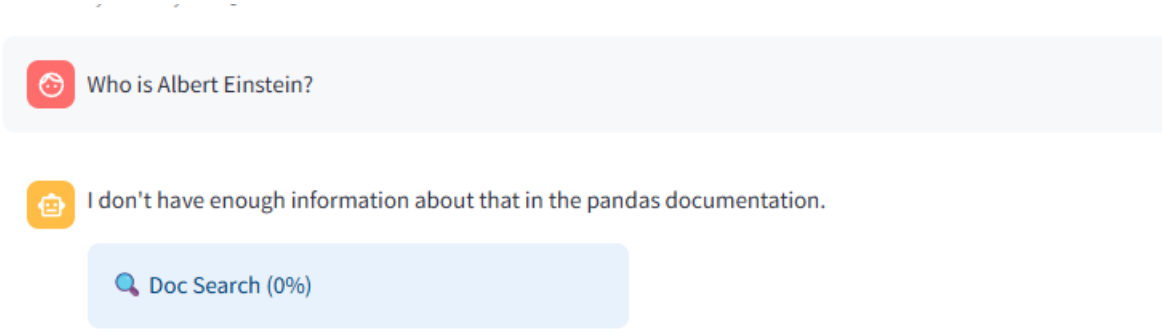
Screenshot 3- QA Answer Example



Screenshot 4 – Doc Search Answer Example



Screenshot 5 – Out-of-scope refusal



7. Limitations & Future Improvements

7.1 Current Limitations

Limitation	Impact	Severity
Static dataset	Knowledge does not update automatically when pandas releases a new version.	Medium
CPU-only inference	Local Ollama inference is slower than GPU or cloud APIs	Medium
Fixed threshold (0.6)	Set empirically; a different threshold may be optimal for code-heavy questions.	Low
No reranking	The doc store returns top-5 chunks by relevance with no further reranking.	Low–Medium
In-memory history	Chat history is lost on page refresh; no persistent sessions.	Low
Scope: User Guide only	API Reference, Release Notes and Changelog are not included.	Medium

7.2 Future Improvements

- Automated re-scraping to keep the dataset current with new pandas releases.
- RAGAS evaluation framework to measure answer faithfulness, relevance and context precision.
- Cross-encoder reranking for better doc-store precision.
- BM25 + vector fusion (Reciprocal Rank Fusion) for improved hybrid retrieval.
- Persistent chat sessions using SQLite.
- Expanded coverage to include pandas API Reference pages.

8. Learnings

8.1 Technical Learnings

Data quality is the dominant factor

The scraping and Q&A-generation pipeline consumed more time and effort than the chatbot logic itself. Clean, well-structured documents produce better embeddings and more relevant retrievals.

Hybrid retrieval value

The two-store hybrid approach proved its value in testing. For frequent, textbook questions the Q&A store provides polished, pre-validated context; for complex or novel questions the doc store + LLM path provides flexible, context-grounded answers. Neither approach alone would have been as effective.

LLM quirks with JSON output

Small local models are inconsistent at producing valid JSON, especially when context chunks contain code blocks or special characters. Key lessons: always implement retry logic; strip markdown fences; and reducing the pairs-per-chunk count and chunk size (3,000 → 1,500 chars) significantly improved JSON reliability.

Multi-query expansion improves recall

Asking the LLM to rephrase the user query into documentation-style variants before searching the Q&A store noticeably increased the qa_match hit rate for colloquial phrasings, at the cost of one short extra LLM call per query.

Embedding model selection matters

multi-qa-MiniLM-L6-cos-v1 outperformed the general-purpose all-MiniLM-L6-v2 for this task because it is specifically optimised for question-answer semantic search, leading to higher Q&A-match precision.

Appendix – Sample Data

A.1 Sample Q&A Pairs

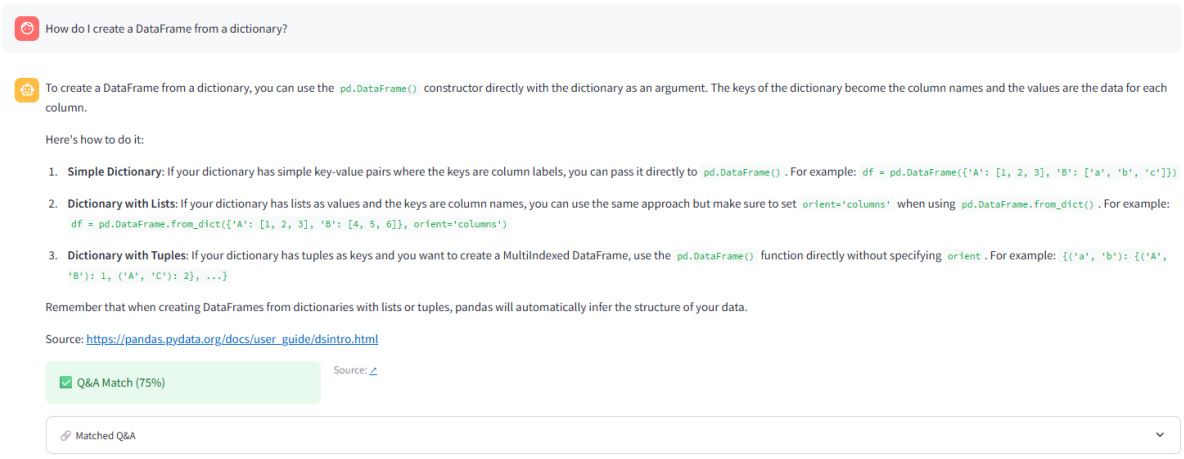
Question	Answer	Source
How do you create a DataFrame from a dictionary?	Pass a dictionary of lists to pd.DataFrame(). Each key becomes a column name and the corresponding list becomes the column values.	10min.html
What is the purpose of the loc accessor?	loc is used for label-based indexing, selecting rows and columns by their index labels. It is inclusive on both ends of a slice.	indexing.html
How does groupby work in pandas?	groupby splits a DataFrame into groups based on one or more columns, applies a function to each group, and combines the results.	groupby.html
What is the difference between merge and concat?	merge combines DataFrames on shared column values (SQL join); concat stacks DataFrames along an axis without matching on values.	merging.html
How do you handle missing values in pandas?	dropna() removes rows with NaN; fillna(value) replaces NaN; isna() detects missing values as a boolean mask.	missing_data.html

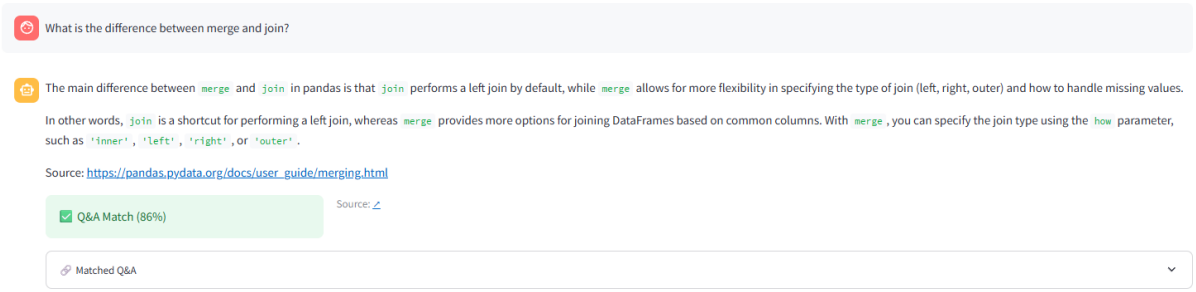
A.2 Data Files Location

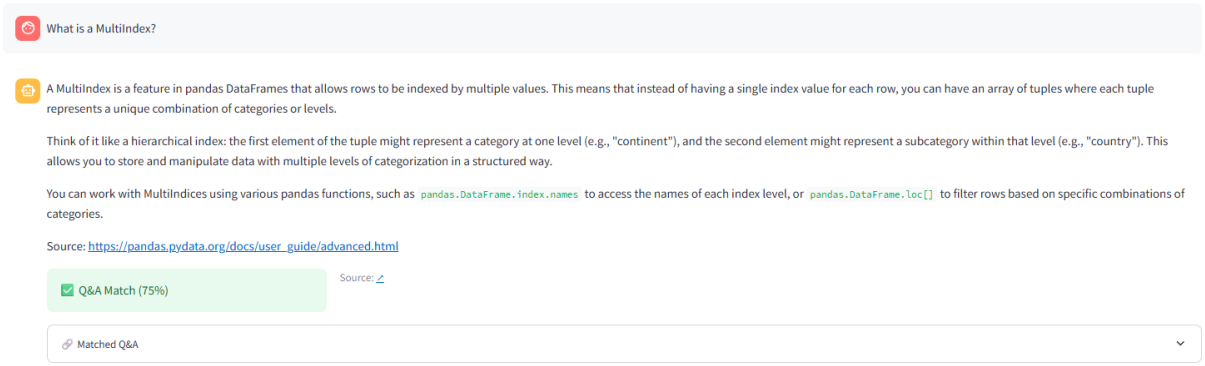
Artifact	Path	Description
Raw pages	data/raw/*.txt	23 cleaned text files, one per scraped page
Q&A dataset	data/processed/qa_dataset_ollama.csv	Q&A pairs (question, answer, source_page)

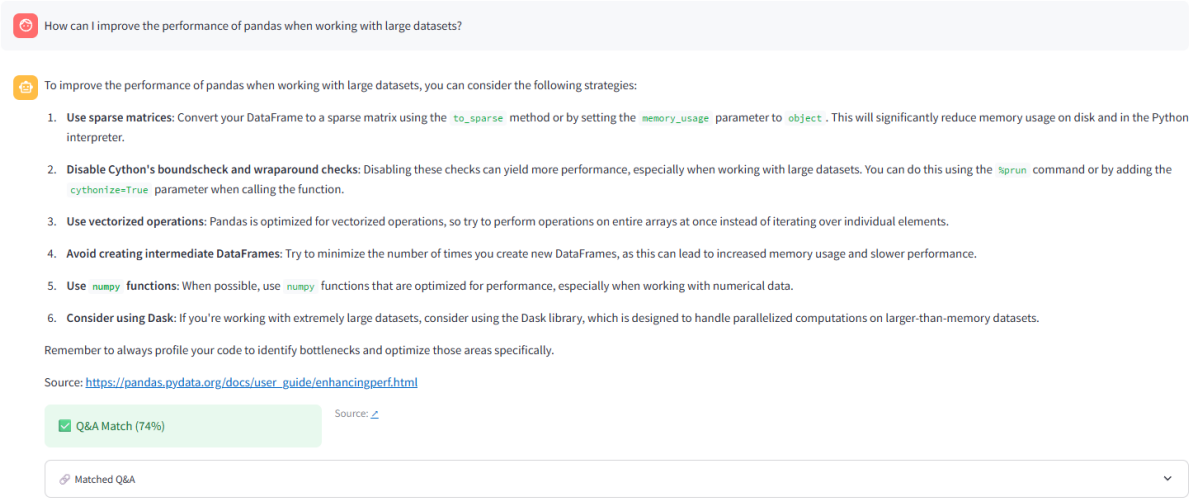
Vector store	chroma_db/	ChromaDB persistent storage (pandas_qa + pandas_docs)

A.3 Further Chatbot Screenshots









A.4 Technology Stack

Python 3.10+ · LangChain · ChromaDB · Streamlit · Ollama llama3.1:8b · multi-qa-MiniLM-L6-cos-v1 · BeautifulSoup4 · sentence-transformers · python-dotenv · pandas · tqdm · requests · lxml